Python: Comprehensive Theory and Concepts Study Guide

Module 1: Python Fundamentals

1. Introduction to Python

- **What is Python?** Python is an interpreted, high-level, general-purpose, dynamically typed programming language. It is designed to be highly readable, using indentation instead of curly braces or semicolons. It supports multiple programming paradigms, including object-oriented, procedural, and functional programming.
- **History of Python** Created by Guido van Rossum in the late 1980s at CWI (Centrum Wiskunde & Informatica) in the Netherlands. It was conceived as a successor to the ABC programming language. Python 1.0 was released in 1991. Python 2.0 (released in 2000) introduced garbage collection and list comprehensions. Python 3.0 (released in 2008) was a backward-incompatible cleanup of the language.
- **Features of Python**
    - **Simple and Readable**: Clean syntax resembling pseudo-code.
    - **Interpreted**: Code is compiled to bytecode and then executed line-by-line at runtime.
    - **Dynamically Typed**: Variable types do not need to be declared explicitly; types are checked at runtime.
    - **Extensible**: Can easily interface with C/C++ libraries.
    - **Batteries Included**: Comes with a massive, feature-rich standard library.
- **Advantages**
    - Rapid prototyping and faster time-to-market.
    - Strong community support and a massive ecosystem of packages (PyPI).
    - Cross-platform availability (Windows, macOS, Linux).
    - Great integration with machine learning, data engineering, and automation tools.
- **Disadvantages**
    - Slower execution speeds compared to statically compiled languages (e.g., C, C++, Rust).
    - High memory consumption due to object wrapper overhead.
    - Weak in native mobile application development.
    - Runtime type errors due to lack of compile-time type checks.
- **Applications of Python**
    - Data Science, Machine Learning, and Artificial Intelligence (NumPy, Pandas, PyTorch).

    - Web Development (Django, FastAPI, Flask).
    - Scripting, Automation, and DevOps (Fabric, Ansible).
    - Web Scraping (BeautifulSoup, Scrapy).
    - APIs, Microservices, and Cloud Infrastructure.

- **Python Versions** Python 2.x and Python 3.x are the two main historical branches. Python 2.x was retired on January 1, 2020. Python 3.x introduced changes such as treating `print` as a function, forcing Unicode by default for strings, and altering division `/` to always yield floats.
- **Python Interpreter** A engine that executes source code directly. The reference implementation is **CPython** (written in C). Other interpreters include **PyPy** (uses a Just-In-Time compiler for speed), **Jython** (runs on JVM), and **IronPython** (runs on .NET).
- **Python Virtual Machine (PVM)** A core component of the interpreter. The PVM is a stack-based execution loop that processes compiled bytecode instructions and converts them into machine code instructions.
- **Bytecode (.pyc)** An intermediate, platform-independent representation of source code compiled by Python. When a `.py` file is imported or run, Python generates a compiled `.pyc` file inside the `__pycache__` folder to speed up future loading times.
- **Compilation vs Interpretation** Python utilizes a hybrid compiler-interpreter model. When a script runs, Python first compiles the source code into bytecode (`.pyc`). The Python Virtual Machine (PVM) then interprets this bytecode line-by-line into machine-specific instructions at runtime.

2. Variables

- **Variable** A variable in Python is a named reference (label) pointing to an object in memory. Variables are stored on the stack and point to objects allocated on the private heap.
- **Variable Naming Rules**
    - Must start with a letter (a-z, A-Z) or an underscore (`_`).
    - Cannot start with a digit.
    - Can only contain alphanumeric characters and underscores (`a-z` , `A-Z` , `0-9` , and `_`).
    - Cannot match Python reserved keywords (like `def` , `class` , `import` , `global`).
- **Dynamic Typing** Variables do not have static types; only the objects they point to have types. You can reassign a variable to point to objects of different types during its lifetime: `python x = 10 # x points to an int object x = "hello" # x now points to a str object`
- **Multiple Assignment** Python allows assigning a single value to multiple variables simultaneously, or unpacking multiple values at once: `python a = b = c = 100 # Single assignment x, y, z = 1, 2.5, "A" # Multiple unpacking assignment`
- **Memory Allocation** Variables store memory addresses. When an assignment like `a = 5` occurs, Python checks if an integer object `5` already exists (thanks to integer interning for values `[-5, 256]`). If it does, `a` references the existing object; if not, a new object is created on the heap.
- **Variable Scope** Dictates where a variable is accessible, resolved using the **LEGB Rule**:
    1. **L**ocal: Declared inside a function.

2. **E**nclosing: In an outer enclosing function (nonlocal).
3. **G**lobal: At the top level of the module.
4. **B**uilt-in: Pre-defined names in Python (e.g., `len`, `range`).

3. Data Types

- **Numeric**
  - `int`: Arbitrary-precision integers (e.g., `42`, `-10`).
  - `float`: IEEE 754 double-precision floating-point numbers (e.g., `3.1415`).
  - `complex`: Complex numbers consisting of real and imaginary parts (e.g., `3 + 5j`).
  - `bool`: Booleans `True` and `False` (subclass of `int`, representing `1` and `0`).
- **Sequence**
  - `String` (`str`): Immutable sequence of Unicode code points (e.g., `"hello"`).
  - `List` (`list`): Mutable, ordered sequence of heterogeneous objects (e.g., `[1, "two", 3.0]`).
  - `Tuple` (`tuple`): Immutable, ordered sequence of heterogeneous objects (e.g., `(1, "two", 3.0)`).
  - `Range` (`range`): Immutable sequence of numbers commonly used in loops (e.g., `range(0, 10)`).
- **Set**
  - `set`: Mutable, unordered collection of unique, hashable items (e.g., `{1, 2, 3}`).
  - `frozenset`: Immutable, hashable version of a set.
- **Mapping**
  - `Dictionary` (`dict`): Collection of key-value pairs where keys must be unique and hashable (e.g., `{"a": 1, "b": 2}`).
- **Binary**
  - `bytes`: Immutable sequence of single bytes (values 0-255).
  - `bytearray`: Mutable sequence of single bytes.
  - `memoryview`: Allows access to the internal buffers of objects supporting the buffer protocol without copying.
- **Special**
  - `NoneType`: Represented by the singleton `None`, indicating the absence of a value or null.

4. Operators

- **Arithmetic**: `+`, `-`, `*`, `/` (float division), `//` (floor division), `%` (modulo), `**` (exponentiation).
- **Assignment**: `=`, `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, and bitwise operators like `&=`, `|=`, `^=`.
- **Comparison**: `==` (value equality), `!=` (value inequality), `<`, `>`, `<=`, `>=`.

- **Logical**: `and` (short-circuiting AND), `or` (short-circuiting OR), `not` (logical NOT).
- **Bitwise**: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (left shift), `>>` (right shift).
- **Identity**:
  - `is`: Tests if two references point to the exact same memory address (e.g., `a is b` is equivalent to `id(a) == id(b)`).
  - `is not`: Tests if they point to different addresses.
- **Membership**:
  - `in`: Checks if a value exists in a sequence/collection.
  - `not in`: Checks if it does not exist.
- **Operator Precedence** Operations are evaluated in this order (highest to lowest):
  1. `()` (Parentheses)
  2. `**` (Exponent)
  3. `+x`, `-x`, `~x` (Unary positive, negative, bitwise NOT)
  4. `*`, `/`, `//`, `%` (Multiplication, division, floor division, modulo)
  5. `+`, `-` (Addition, subtraction)
  6. `<<`, `>>` (Bitwise shifts)
  7. `&` (Bitwise AND)
  8. `^` (Bitwise XOR)
  9. `|` (Bitwise OR)
  10. `==`, `!=`, `<`, `>`, `<=`, `>=`, `is`, `is not`, `in`, `not in` (Comparisons)
  11. `not` (Logical NOT)
  12. `and` (Logical AND)
  13. `or` (Logical OR)

5. Input Output

- **`input()`** Reads a line of input from standard input, strips the trailing newline, converts it to a string, and returns it. `python age = int(input("Enter your age: ")) # Explicit cast to integer`
- **`print()`** Prints objects to a text stream. Signature: `print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`.
- **Formatting**
  - **f-string (Formatted String Literals)**: Evaluated at runtime; highly readable and performant. `python name = "Bob" print(f"Hello, {name.lower()}! Double age: {2 * 12}")`
  - **`format()` method**: `python print("Hello, {}. You are {}.".format("Alice", 25))`
  - **Escape Characters**: `\n` (newline), `\t` (tab), `\\` (backslash), `\'` (single quote), `\"` (double quote).

Module 2: Strings

- **String** An immutable sequence of Unicode characters. Any operation that modifies a string returns a brand-new string object.
- **String Indexing** Zero-based from the left, negative indexing from the right: `python s = "Python" # s[0] is 'P', s[-1] is 'n'`
- **Slicing** Extracts a range of characters. Syntax: `string[start:stop:step]`. Note that `stop` is exclusive. `python s = "Python" print(s[0:3]) # "Pyt" print(s[::-1]) # "nohtyP" (reverses the string)`
- **Immutability** You cannot modify individual characters directly. Doing `s[0] = 'J'` raises a `TypeError`.
- **String Functions**
  - `split(sep=None)`: Splits a string into a list of substrings using a separator.
  - `join(iterable)`: Joins elements of an iterable of strings using the string as a separator.
  - `replace(old, new)`: Returns a copy with all occurrences of `old` replaced by `new`.
  - `strip()` / `lstrip()` / `rstrip()`: Removes leading and trailing whitespace or custom characters.
  - `startswith(prefix)` / `endswith(suffix)`: Returns `True` if the string starts/ends with the specified value.
  - `find(sub)`: Returns the lowest index where substring `sub` is found, or `-1` if not found.
  - `count(sub)`: Returns the number of non-overlapping occurrences of `sub`.
- **String Formatting** Supports `%` format specifiers, `.format()`, and modern f-strings.
- **Raw String** Prefixed with `r` or `R`. Causes backslashes (`\`) to be treated as literal, raw characters instead of escape characters: `python path = r"C:\new_folder\text.txt" # No escape sequences processed`

Module 3: Collections

1. List

- **Creation**: Using square brackets `[]` or `list()`.
- **Indexing & Slicing**: Identical to strings, but elements can be modified.
- **Mutable**: Elements can be replaced, updated, or added in-place.
- **List Methods**:
  - `append(x)`: Adds an item to the end of the list ($O(1)$ amortized).
  - `extend(iterable)`: Appends elements from an iterable to the list.
  - `insert(i, x)`: Inserts an item at a given index ($O(N)$).
  - `remove(x)`: Removes the first item with value `x` ($O(N)$).

    - `pop(i)`: Removes and returns item at index `i` (defaults to last element, $O(1)$ for last, $O(N)$ otherwise).
    - `sort(key=None, reverse=False)`: Sorts the list in-place using the Timsort algorithm.
    - `reverse()`: Reverses list elements in-place.

- **Nested List**: Lists containing other lists. Used to represent matrices.
- **List Comprehension**: A concise way to generate lists. `python squares = [x**2 for x in range(10) if x % 2 == 0]`
- **Copy**
    - **Shallow Copy**: Creates a new list container, but copies references to the nested elements. Modifying nested mutable objects affects both copies. E.g., `lst.copy()` or `copy.copy(lst)`.
    - **Deep Copy**: Recursively copies both the container and all nested elements. Modifying nested objects in the copy has no effect on the original. E.g., `copy.deepcopy(lst)`.

2. Tuple

- **Creation**: Declared using parentheses `()` or comma-separated values. Single-item tuples need a trailing comma: `t = (5,)`.
- **Packing**: Assigning multiple values to a tuple without parentheses: `t = 1, 2, "three"`.
- **Unpacking**: Binding individual variables to items inside a tuple: `x, y, z = t`. Can use `*` to capture remainder: `a, *b = (1, 2, 3, 4)`.
- **Immutable**: Once created, its elements cannot be altered or reassigned.
- **Tuple Methods**: `count(x)` and `index(x)`.
- **Advantages**:
    - Faster iteration and access speeds than lists.
    - Guarantees write-protection for data.
    - Can be used as keys in dictionaries (since they are hashable, provided all elements inside are hashable).

3. Set

- **Creation**: Set literals `{1, 2, 3}` or `set()`. Empty sets must be created using `set()`, as `{}` creates an empty dictionary.
- **Set Operations**:
    - **Union (`|` or `union()`)**: All unique elements from both sets.
    - **Intersection (`&` or `intersection()`)**: Elements common to both sets.
    - **Difference (`-` or `difference()`)**: Elements in the left set but not the right.
    - **Symmetric Difference (`^` or `symmetric_difference()`)**: Elements in either set, but not both.
- **Frozen Set**: An immutable set created with `frozenset()`. It is hashable, meaning it can be added to other sets or used as dictionary keys.

4. Dictionary

- **Key-Value Pair**: A mapping implementation. Keys must be unique and hashable. Values can be of any type.
- **Dictionary Methods**:
  - `keys()`: Returns a view object of keys.
  - `values()`: Returns a view object of values.
  - `items()`: Returns a view object of key-value tuples.
  - `get(key, default=None)`: Returns value for `key`, or `default` if key doesn't exist (prevents `KeyError`).
  - `update(other)`: Merges another dictionary or key-value iterable into it in-place.
  - `pop(key)`: Removes and returns the value of the specified key.
  - `popitem()`: Removes and returns the last inserted key-value pair ($O(1)$).
- **Nested Dictionary**: A dictionary where values themselves are dictionaries.
- **Iteration**: Loop through keys directly (`for k in dict`), or unpack items (`for k, v in dict.items()`).
- **Dictionary Comprehension**: `python char_counts = {char: s.count(char) for char in set(s)}`

Module 4: Control Statements

- **if, if-else, elif, Nested if** Conditional execution blocks. Conditionals evaluate truthiness (empty collections, `0`, `None`, and `False` are falsy; all other values are truthy).
- **for** Iterates over an iterable (like list, tuple, range, string).
- **while** Repeats a block of code as long as its condition remains `True`.
- **break** Terminates the innermost loop immediately, skipping the loop's optional `else` block.
- **continue** Skips the rest of the current iteration and jumps to the next iteration of the loop.
- **pass** A null statement that serves as a syntax placeholder where code is syntactically required but no action needs to be taken.
- **match-case (Structural Pattern Matching)** Introduced in Python 3.10. Replaces complex `if-elif` blocks with pattern matching: `python match status_code: case 200: return "OK" case 404: return "Not Found" case 500 | 503: return "Server Error" case _: return "Unknown Status"`

Module 5: Functions

- **Function** A named, reusable block of code defined with `def` that executes only when called.
- **Parameters vs Arguments**

    - **Parameters**: The variables listed in the function definition (e.g., `def add(x, y):`).
    - **Arguments**: The actual values passed to the function when it is invoked (e.g., `add(5, 10)`).

- **Return** Exits a function and returns a value to the caller. If no `return` statement is written, or a bare `return` is executed, the function implicitly returns `None`.
- **Default Arguments** Allows parameters to have default values if no argument is passed. > [!IMPORTANT] > Never use mutable objects (like lists or dictionaries) as default arguments, because defaults are evaluated only once at definition time and shared across all calls. ```python # Bad def append_to(element, target=[]): target.append(element) return target

Good

def append_to(element, target=None): if target is None: target = [] target.append(element) return target `* **Keyword Arguments** Passing arguments by matching parameters by name, allowing arguments to be passed in any order: `func(age=25, name="Alice")`. * **Positional Arguments** Arguments matched strictly by their position order: `func("Alice", 25)`. * **Variable-Length Arguments** * `*args`: Collects extra positional arguments into a **tuple**. * `**kwargs`: Collects extra keyword arguments into a **dictionary**.` python def dump_args(*args,* *kwargs): print(args) # e.g., (1, 2) print(kwargs) # e.g., {'x': 10} ``` `* **Lambda Function** Small, single-expression anonymous functions. Syntax:` lambda arguments: expression `. E.g.,` add = lambda x, y: x + y `. * **Recursive Function** A function that calls itself to solve smaller sub-problems. Must have a base case to terminate execution. * **Nested Function** A function defined inside another function. It can access variables defined in the outer scope (closure mechanism). * **Higher-Order Function** A function that accepts other functions as arguments, returns a function, or both (e.g.,` map() `,` filter() `,` sorted()`).

Module 6: Functional Programming

- **Lambda** Often used inline inside higher-order functions to avoid writing dedicated functions for one-off operations.
- **`map(func, iterable)`** Applies the function `func` to every item of the `iterable` and returns a lazy map iterator object.
- **`filter(func, iterable)`** Filters items from the `iterable` for which the function `func` returns `True`. Returns a lazy filter iterator.
- **`reduce(func, iterable[, initializer])`** Imported from `functools`. Applies a function of two arguments cumulatively to the sequence, reducing it to a single value.
- **`zip(*iterables)`** Aggregates elements from each of the iterables in parallel, returning an iterator of tuples. Stops when the shortest iterable is exhausted (unless using `itertools.zip_longest`).

- **`enumerate(iterable, start=0)`** Takes an iterable and returns an iterator of tuples containing the index (starting from `start`) and the corresponding value.
- **`any(iterable)`** Returns `True` if at least one element in the iterable evaluates to truthy; returns `False` if empty.
- **`all(iterable)`** Returns `True` if all elements in the iterable evaluate to truthy (or if the iterable is empty).

Module 7: Modules & Packages

- **Module** A file containing Python definitions and statements (extension `.py`). Used to break down large programs.
- **Package** A directory containing modules. Packages allow hierarchical structuring. Historically required an `__init__.py` file in the folder to be treated as a package (optional since Python 3.3 namespace packages).
- **import, from-import, and alias**
  - `import math`: Imports entire module namespace.
  - `from math import sqrt`: Imports specific function into the local namespace.
  - `import pandas as pd`: Imports module with an alias.
- **`__name__` and `__main__`** `__name__` is a built-in variable. If a script is run directly, `__name__` is set to `"__main__"`. If the module is imported, `__name__` is set to the module's file name. Used as a gatekeeper block: `python if __name__ == "__main__": # Execute only when run directly main()`
- **pip** The standard package installer for Python, used to download packages from PyPI (Python Package Index).
- **Virtual Environment** An isolated environment tool (created via `python -m venv env_name`) that manages dependency versions on a per-project basis, preventing version conflicts with globally installed libraries.

Module 8: File Handling

- **File** A named location on disk used to store persistent data.
- **Modes**
  - `'r'`: Read (default). Fails if file does not exist.
  - `'w'`: Write. Overwrites existing files or creates a new file.
  - `'a'`: Append. Appends data to the end of the file.
  - `'rb'`, `'wb'`: Read/Write in binary mode.
  - `'+'`: Open for updating (read/write). e.g., `'r+'`.
- **Read**

    - `read(n)`: Reads `n` characters/bytes (reads whole file if unspecified).
    - `readline()`: Reads a single line.
    - `readlines()`: Reads all lines and returns them as a list of strings.
- **Write**
    - `write(string)`: Writes a string to the file and returns characters written.
    - `writelines(list_of_strings)`: Writes a list of strings to the file.
- **Append** Opens file and places pointer at the end. Writes do not alter existing text.
- **Binary File vs Text File**
    - **Text File**: Stores characters encoded in a standard format (e.g., UTF-8). Handles platform newline conversions (`\r\n` to `\n`).
    - **Binary File**: Stores raw bytes directly without any encoding/decoding. Necessary for images, zip files, or executables.
- **CSV** Handled by the built-in `csv` module: `python import csv with open("data.csv", "r") as f: reader = csv.reader(f) for row in reader: print(row)`
- **JSON** Handled by the built-in `json` module:
    - `loads()` / `dumps()`: Serializes/deserializes to and from strings.
    - `load()` / `dump()`: Reads/writes JSON directly to file streams.
- **with Statement** A cleaner way to manage files. It automatically closes the file stream when the block exits, even if exceptions are raised.

Module 9: Exception Handling

- **Exception** An event triggered during program execution that disrupts the normal flow of instructions.
- **try, except, else, finally** `python try: result = 10 / divisor except ZeroDivisionError as e: print("Cannot divide by zero:", e) else: print("Division succeeded. Result is:", result) # Runs only if no exception occurred finally: print("Execution complete.") # Always runs`
- **raise** Forces a specified exception to occur. E.g., `raise ValueError("Invalid entry")`.
- **assert** Used for debugging. Evaluates a condition. If the condition is `False`, it raises an `AssertionError`. `python assert age ⩾ 0, "Age cannot be negative"`
- **Custom Exception** Defined by creating a class that inherits from Python's built-in `Exception` class: `python class InvalidAgeError(Exception): pass`

Module 10: OOP (Object-Oriented Programming)

1. Basics

- **Class**: A user-defined prototype/blueprint for creating objects.
- **Object**: An instance of a class that encapsulates state (attributes) and behavior (methods).
- **Instance Variable**: Variables unique to each instance, defined inside methods using `self.variable_name`.
- **Class Variable**: Variables shared by all instances of a class. Defined directly in the class scope.
- **Constructor**: The `__init__` method, invoked automatically during instantiation.
- **Destructor**: The `__del__` method, called when the object's reference count drops to zero (deallocated).
- **self**: A reference to the current instance of the class, used to access instance variables and methods.
- **cls**: A reference to the class itself, used in class methods.

2. Four Pillars of OOP

- **Encapsulation** Restricts direct access to some of an object's components. In Python, access modifiers are based on naming conventions:
  - **Public**: Access from anywhere (`self.name`).
  - **Protected**: Access within class and subclasses. Prefixed with a single underscore (`self._name`).
  - **Private**: Access only within the class. Prefixed with a double underscore (`self.__name`). Triggers **Name Mangling**, renaming it to `_ClassName__name` under the hood.

- **Abstraction** Hides complex details and exposes only essential interfaces. Achieved using the `abc` module: ```python from abc import ABC, abstractmethod

  class Vehicle(ABC): @abstractmethod def start_engine(self): pass `* **Inheritance** Enables a class to inherit attributes and methods from another class. * **Single**: Subclass inherits from one superclass. * **Multiple**: Subclass inherits from more than one superclass. * **Multilevel**: Subclass inherits from another subclass. * **Hierarchical**: Multiple subclasses inherit from a single superclass. * **Hybrid**: Mix of multiple types. * **`super()`**: Returns a proxy object to call methods of parent classes, resolving execution order via Method Resolution Order (MRO) using C3 Linearization. * **Polymorphism** Allows different classes to be treated through the same interface. * **Method Overriding**: Subclass provides a specific implementation of a method declared in the parent class. * **Method Overloading**: Writing multiple methods with the same name but different parameters. Python does *not* support compile-time method overloading; the latest method definition replaces previous ones. Overloading is simulated using default arguments or variable-length arguments`

(`*args`, `**kwargs`). * **Duck Typing**: If it looks like a duck and quacks like a duck, it is treated as a duck. Python prioritizes interface behavior over formal class inheritance: python def fly_object(flyable): flyable.fly() # Calls fly method regardless of class type ```

Module 11: Advanced OOP

- **Static Method** Declared with the `@staticmethod` decorator. Does not accept an implicit first argument (`self` or `cls`). Behaves like a standard utility function grouped inside a class.
- **Class Method** Declared with the `@classmethod` decorator. Receives the class (`cls`) as the first argument. Often used as alternative constructors (factory methods).
- **Property Decorator** Provides getters, setters, and deleters for instance variables, allowing attributes to be accessed like variables but validated via method calls: ```python class Employee: def **init**(self, salary): self._salary = salary

```
@property
def salary(self):
    return self._salary

@salary.setter
def salary(self, value):
    if value < 0:
        raise ValueError("Salary cannot be negative")
    self._salary = value
```

`` * **Magic Methods / Dunder Methods** Special methods prefixed and suffixed with double underscores (e.g., **init**, **str**, **repr**, **len**, **getitem**). They define custom behavior for built-in operations (like arithmetic, indexing, or string conversion). * **Operator Overloading** Allows customization of how standard operators (like +, -, *, ==) behave when applied to user-defined objects. Defined using specific dunder methods: * + `calls` **add**(self, other) * == `calls` **eq**(self, other) * < `calls` **lt**(self, other)`

Module 12: Iterators

- **Iterable** An object capable of returning its members one at a time. It implements the `__iter__()` method, which must return an iterator. Examples: lists, tuples, dictionaries, sets.
- **Iterator** An object representing a stream of data. It implements:
  - `__next__()`: Returns the next item. Raises `StopIteration` when no more elements remain.
  - `__iter__()`: Returns the iterator object itself.

- **`iter()`** The built-in function that calls the `__iter__()` method of an iterable to obtain an iterator.
- **`next()`** The built-in function that calls the `__next__()` method of an iterator to fetch the next element.
- **StopIteration** An exception automatically raised by `__next__()` to signal that all elements have been exhausted, halting loops like `for`.

Module 13: Generators

- **Generator** A special kind of function that simplifies iterator creation. Instead of returning a single value and exiting, a generator yields values one at a time using `yield`, pausing its execution state after each yield.
- **`yield`** A keyword that returns a value to the caller and temporarily suspends the function's execution state, keeping local variables intact. When execution resumes, it starts immediately after the `yield` statement.
- **Generator Expression** A memory-efficient shorthand for generating generator objects. Syntax is similar to list comprehensions but uses parentheses: `python gen = (x**2 for x in range(1000000)) # Lazy evaluation, consumes almost zero memory`
- **Generator vs Iterator**
  - All generators are iterators, but not all iterators are generators.
  - Generators are written as functions with `yield` statements (or expressions).
  - Iterators are written by explicitly creating classes implementing `__iter__` and `__next__`.
  - Generators track local state automatically; custom iterators require manual instance variables.

Module 14: Decorators

- **Decorator** A design pattern that allows adding new behavior to an existing callable (function or class) without permanently modifying its structure. It wraps the target function.
- **Function Decorator** A function that takes another function as an argument, defines a wrapper, and returns the wrapper. ```python def my_decorator(func): def wrapper(*args, kwargs): print("Before call") res = func(*args, **kwargs) print("After call") return res return wrapper

  @my_decorator def say_hello(): print("Hello!") `* **Nested Decorator** Multiple decorators applied to a single function. They execute in bottom-up/inside-out order:` python @dec1 @dec2 def my_func(): pass

Equivalent to: my_func = dec1(dec2(my_func))

`* **Parameterized Decorator** A decorator that accepts arguments. Requires three levels of nested functions:` python def repeat(num_times): def decorator_repeat(func): def wrapper(*args, kwargs): for _ in range(num_times): res = func(*args, **kwargs) return res return wrapper return decorator_repeat `` * ** **@property`** A built-in decorator used to define property attributes (getters, setters, deleters) within a class namespace.

Module 15: Context Manager

- **Context Manager** An object that runtime-manages resources (like database connections, file handles, or network sockets) by defining setup and teardown phases.
- **with Statement** The syntax construct that invokes a context manager: `python with open("file.txt", "r") as f: data = f.read() # File is guaranteed to be closed here even if f.read() threw an exception`
- **`__enter__()` and `__exit__()`** To create a custom context manager class, implement:
  - `__enter__(self)`: Sets up resources and returns the resource object (bound to `as var`).
  - `__exit__(self, exc_type, exc_val, exc_tb)`: Tears down resources. If an exception occurred, it receives the error details. If it returns `True`, the exception is suppressed; if it returns `False` / `None`, the exception propagates.
- **Alternative using contextlib** ```python from contextlib import contextmanager

  @contextmanager def manage_resource(): print("Setup resource") yield "Resource" print("Teardown resource") ```

Module 16: Memory Management

- **Reference Counting** Python's primary memory management system. Each object tracks how many variables, containers, or functions reference it. When an object's reference count drops to 0, its memory is immediately deallocated.
- **Garbage Collection (Cyclic Garbage Collector)** Reference counting cannot clean up **reference cycles** (e.g., Object A references Object B, and Object B references Object A). Python's cyclic garbage collector (`gc` module) runs periodically to detect and destroy these cycles. It operates using three **generations** (0, 1, 2). New objects start in Generation 0; if they survive a GC collection, they are promoted to Generation 1, and eventually Generation 2. Older generations are collected less frequently.

- **Memory Allocation** Python manages a private heap containing all Python objects. The **PyMalloc** allocator is used for small objects (up to 512 bytes) to avoid overhead from the system allocator.
- **`del` Keyword** Deletes a reference to an object, removing the variable name from its namespace and decrementing the object's reference count by 1. > [!WARNING] > `del` does not directly delete objects or free memory; it only deletes variable names and reduces reference counts. Deallocation happens when reference counts hit 0.

Module 17: Copying Objects

- **Assignment** Doing `y = x` does not copy the object. It copies the reference. Both `y` and `x` point to the exact same memory address (`id(x) == id(y)`).
- **Shallow Copy** Copies the outer container object but copies references to any nested objects. Modifying nested mutable objects impacts both the original and copy.
  - Using `copy.copy(obj)` or list slicing `lst[:]` or dict `.copy()`.
- **Deep Copy** Recursively copies the container *and* all objects nested inside it. Creates entirely independent duplicates.
  - Using `copy.deepcopy(obj)`.
- **`copy` Module** The built-in Python module providing standard shallow (`copy()`) and deep (`deepcopy()`) replication operations.

Module 18: Multithreading

- **Thread** The smallest unit of execution context within a single process. Multiple threads share the same process memory space.
- **Thread Class** Implemented in Python using the `threading.Thread` class: ```python import threading

  def print_numbers(): for i in range(5): print(i)

  t = threading.Thread(target=print_numbers) t.start() t.join() # Waits for the thread to complete `` * **Thread Lifecycle** 1. **New**: Thread is created. 2. **Runnable**: start() is called; thread is ready to run. 3. **Running**: Thread is executing instructions. 4. **Blocked/Waiting**: Thread is waiting for I/O, locks, or sleeping. 5. **Terminated**: Thread completes execution. * **Synchronization** Used to prevent **race conditions** (multiple threads writing/reading shared state simultaneously). * **Lock** A primitive lock (mutex) with two states: locked and unlocked. A thread acquires it (lock.acquire()), performs work, and releases it (lock.release()`). * **RLock (Reentrant Lock)** A lock that can be acquired multiple times by the *same* thread without causing self-deadlock. Must be released the same number of times it

was acquired. * **Semaphore** A counter-based lock that limits concurrent access to a resource to a set maximum number of threads. * **Deadlock** A situation where two or more threads are blocked indefinitely, each waiting for a lock held by another thread.

Module 19: Multiprocessing

- **Process** An independent execution unit with its own virtual memory space, interpreter, and GIL. Because processes do not share memory by default, multiprocessing bypasses the GIL and scales CPU-bound tasks.
- **Pool** Provides a pool of worker processes to parallelize execution of a function across multiple input values (`multiprocessing.Pool`).
- **Inter-Process Communication (IPC)**
  - **Queue**: A multi-producer, multi-consumer FIFO queue that is process-safe.
  - **Pipe**: A communication channel between two processes (returns a tuple of connection endpoints).
  - **Shared Memory**: Shared state containers using `multiprocessing.Value`, `multiprocessing.Array`, or the Python 3.8+ `shared_memory` module.

Module 20: GIL (Global Interpreter Lock)

- **What is GIL?** A mutex (lock) in the standard CPython interpreter that ensures only one thread executes Python bytecode at a time, even on multi-core systems.
- **Why Python has GIL?**
  - Protects CPython's memory management (which is not thread-safe by default due to reference counting).
  - Simplifies C extensions integration, as C libraries do not have to handle complex concurrent execution contexts.
  - Makes single-threaded code extremely fast and simple.
- **Advantages**
  - Fast single-threaded execution.
  - Simple memory management.
  - Easy implementation of C modules.
- **Disadvantages**
  - CPU-bound multithreaded applications cannot run in parallel across multiple CPU cores.
- **GIL vs Multithreading**
  - In **I/O-bound tasks** (e.g., API requests, file writes, database queries), threads release the GIL while waiting for the OS, allowing multithreading to speed up the process.

    - In **CPU-bound tasks**, multithreading yields no speedup (and may run slower due to context switching overhead) because only one thread can run at a time.

- **GIL vs Multiprocessing** To scale CPU-bound tasks, use multiprocessing. Each process has its own Python interpreter and GIL, achieving true multi-core parallel execution.

Module 21: Python Internals

- **Bytecode** The low-level instruction set generated by Python when compiling `.py` files. It is an abstraction layer that PVM runs.
- **PVM** The bytecode execution engine that runs compiled bytecode.
- **Compilation Process**
    1. **Parse**: Source code is read and converted to an Abstract Syntax Tree (AST).
    2. **Compile**: AST is compiled into bytecode.
    3. **Interpret**: PVM interprets bytecode into machine code.
- **Memory Management** Separates memory into the **Stack** (stores variables, active function call frames, and references) and the **Heap** (stores actual objects).
- **Namespace** A dictionary mapping names (keys) to objects (values).
- **Scope** The textual region of a Python program where a namespace is directly accessible.
- **LEGB Rule** The order in which namespaces are searched during name resolution: `Local → Enclosing → Global → Built-in`

Module 22: Advanced Python

- **Comprehensions**
    - **List**: `[x for x in range(5)]`
    - **Dictionary**: `{x: x**2 for x in range(5)}`
    - **Set**: `{x for x in range(5)}`
- **Walrus Operator (`:=`)** Introduced in Python 3.8. Allows variable assignment inside expressions: `python if (n := len(data)) > 10: print(f"List is too long ({n} items)")`
- **Type Hinting** Allows annotating code with expected types to improve readability and catch bugs via external tools like `mypy`: `python def greet(name: str) → str: return f"Hello {name}"`
- **Dataclasses** Introduced in Python 3.7. Automates template code generation for classes storing data: ```python from dataclasses import dataclass

  @dataclass class User: id: int username: str

Automatically generates init, repr, eq, etc.

`* **NamedTuple** A subclass of tuple with named fields, combining tuple immutability with named attribute access:` python from typing import NamedTuple

class Point(NamedTuple): x: int y: int `` `* **Collections Module** *` Counter `: Dict subclass for counting hashable objects. *` defaultdict `: Dict subclass that calls a factory function to supply missing values. *` OrderedDict `: Dict subclass that remembers key insertion order (less vital since standard dicts preserve order). *` deque `: Double-ended queue for fast $O(1)$ appends and pops on both ends. *` ChainMap`: Groups multiple dictionaries into a single dictionary-like view.

Module 23: Important Built-in Functions

- **`len(s)`**: Returns the number of elements in an object.
- **`id(obj)`**: Returns the unique integer identifier of an object (its memory address in CPython).
- **`type(obj)`**: Returns the type object of `obj`.
- **`isinstance(object, classinfo)`**: Returns `True` if the object is an instance or subclass of `classinfo`.
- **`sorted(iterable, key=None, reverse=False)`**: Returns a new sorted list from the items in the iterable.
- **`sum(iterable, start=0)`**: Sums the start value and the items of an iterable.
- **`max(arg1, arg2, *args, key=None)`** / **`min()`**: Returns the largest/smallest item in an iterable.
- **`abs(x)`**: Returns the absolute value of a number.
- **`round(number, ndigits=None)`**: Rounds a number to `ndigits` precision after the decimal point.
- **`range(start, stop[, step])`**: Generates a sequence of numbers.
- **`open(file, mode='r')`**: Opens a file and returns a file object stream.
- **`dir([object])`**: Returns a list of valid attributes and methods of the object.
- **`help([object])`**: Invokes the built-in help system.

Module 24: Frequently Asked Interview Comparisons

Comparison	Aspect 1	Aspect 2
List vs Tuple	Lists are mutable ($O(1)$ updates) and slower; used for homogeneous dynamic sequences.	Tuples are immutable, consume less memory, and are faster; used for heterogeneous static data.
List vs Set	Lists are ordered, allow duplicates, and have $O(N)$ lookup.	Sets are unordered, hold only unique elements, and have $O(1)$ average lookup.
Tuple vs Set	Tuples are ordered, immutable, allow duplicates, and are indexed.	Sets are unordered, mutable, forbid duplicates, and cannot be indexed.
Set vs Dictionary	Sets store only unique hashable elements.	Dictionaries store key-value pairs where keys must be unique and hashable.
== vs is	`==` checks value equality (whether values are identical).	`is` checks reference identity (whether they point to the same memory location).
append() vs extend()	`append(x)` adds `x` as a single element to the end of the list.	`extend(iter)` unpacks the iterable and appends all its elements to the list.
remove() vs pop()	`remove(x)` deletes the first item with the value `x`. Raises ValueError if missing.	`pop(i)` removes and returns the item at index `i` (defaults to last element).
copy() vs deepcopy()	`copy()` copies the outer object, keeping shared nested references.	`deepcopy()` recursively copies the entire nested object graph.
Generator vs Iterator	Generators are functions yielding items lazily, holding state automatically.	Iterators are class instances requiring `__iter__` and `__next__` implementation.
Thread vs Process	Threads share memory space and are lightweight; limited by the GIL for CPU tasks.	Processes have isolated memory, require IPC, and run in parallel on multi-core systems.

Comparison	Aspect 1	Aspect 2
Multiprocessing vs Multithreading	Multiprocessing is best for CPU-bound tasks to bypass the GIL.	Multithreading is best for I/O-bound tasks to optimize waiting time.
Method Overloading vs Overriding	Overloading is defining multiple methods with different signatures (simulated in Python).	Overriding is redefining a parent class method in a subclass.
Abstraction vs Encapsulation	Abstraction focuses on *what* the object does (hiding details using interfaces).	Encapsulation focuses on *how* it does it (bundling data and restricting access).
Class Method vs Static Method	Class method receives the class `cls` as an argument; can act as a factory.	Static method acts like a plain function inside class namespace; has no `cls` or `self`.
Local vs Global Variable	Local variables are defined inside functions and destroyed on exit.	Global variables are defined at the module level and persist throughout script execution.
Mutable vs Immutable	Mutable objects (list, dict, set) can change their state/value in-place.	Immutable objects (int, float, str, tuple, frozenset) cannot be modified after creation.
pass vs continue vs break	`pass` is a syntax placeholder that does nothing.	`continue` skips the rest of the current iteration, jumping to the next loop pass.
break	`break` exits the loop entirely.	

📚 Module Summary & Key Takeaways

> **Theory Focus:** Advanced Python Internals, OOP Paradigms, Memory Management, and Concurrency.

This module provides a deep dive into pythonic concepts and language internals. Key operational principles to retain include:

- **Memory Management & GIL:** Python implements automatic memory management via reference counting and a cyclic garbage collector. The **Global Interpreter Lock (GIL)** restricts execution to a single thread per process, meaning multithreading is optimal for I/O-bound operations while multiprocessing is required to bypass the GIL for CPU-bound computations.
- **Object-Oriented Design (OOP):** Structural integrity is maintained via the four OOP pillars: Inheritance, Polymorphism, Encapsulation, and Abstraction. Python determines inheritance paths using the C3 Linearization algorithm to solve Method Resolution Order (MRO).
- **Metaprogramming & Optimization:** Leverages advanced syntax patterns including decorators (to wrap functions/classes), generators (using `yield` for lazy memory-efficient evaluation of large streams), iterators, and context managers (via `__enter__` and `__exit__`) for reliable resource pooling.